WebFGSR

Super Resolution and Frame Interpolation in WebGPU

Hangyu Zhang, Weike Qian, Yikai Li
CIS 5650 GPU Programming and Architecture / Fall 2026

1 Motivation and goals

Rendering fewer pixels and frames saves GPU work, but reconstruction can produce ghosting and unstable edges. We will build a WebGPU super-resolution (SR) and frame-generation (FG) system, drawing on Mob-FGSR [1] and subsequent temporal supersampling research [2].

- **Core:** temporal SR at 2x per axis and one interpolated midpoint between rendered frames. Deliver a browser demo, code, and reproducible experiments.
- **Required quality study:** implement and evaluate depth-consistency and projection-coverage cues near occlusions and thin structures. Test opaque geometry with camera and rigid-object motion.
- **Optional:** selectively re-shade unreliable regions [3] after core completion.

2 Approach

Use low-resolution color, depth, motion vectors, and jitter. WGSL passes will implement history reprojection, motion splatting, visibility resolution, and reconstruction. Reliability cues will guide SR history retention and FG endpoint fusion with task-specific rules. Generated frames will remain separate from SR history. Profile GPU memory access and pass scheduling to guide optimization.

3 Evaluation

Compare spatial upscaling/frame repetition, SR alone, basic SR+FG, and refined SR+FG against full-resolution references at matching simulation times. Report PSNR/SSIM, continuous videos, and cue ablations for occlusion and thin-edge cases. Measure per-pass and total rendering-plus-reconstruction GPU time, memory, and interpolation buffering latency.

4 Milestones

M1 - Nov 11: Validate inputs and reference capture; demonstrate SR and initial measurements.

M2 - Nov 18: Integrate midpoint FG with SR; prototype reliability cues.

M3 - Nov 30: Complete quality refinement, ablations, and GPU optimization.

Final - Dec 6: Complete the demo, code, experiments, and project submission by 11:59pm; prepare for the Dec 7 presentation.

5 Platform and sources

Use TypeScript and Babylon.js WebGPUEngine in desktop Chrome. Babylon.js handles scene rendering; we implement SR/FG and reliability passes in WGSL through ComputeShader. Mob-FGSR/mob-ss may provide adapted code. Document reuse, retain attribution, and obtain course approval before using external implementations.

References

1. *Mob-FGSR: Frame Generation and Super Resolution for Mobile Real-Time Rendering.* SIGGRAPH 2024. Paper/code.

2. *Lightweight, Edge-Aware, and Temporally Consistent Supersampling for Mobile Real-Time Rendering.* SIGGRAPH Asia 2025. Official repository.

3. *Consecutive Frame Extrapolation with Predictive Sparse Shading.* SIGGRAPH Asia 2025. Project.